

Project 2: Formal Criteria-Based Test Design & Automation

Team members:

Rana Ibrahim Shari 2333040

Maria Malek Almomani 2338946

In this project, we are testing a system called the **Premium Discount Eligibility System**.

The system calculates the discount percentage for a customer depending on:

- the customer type
- the number of orders in the last year
- whether the customer is subscribed to the newsletter or not

The class that we will create is: **DiscountCalculator**

The method that will calculate the discount is:

```
calculateDiscount(String customerType, int totalOrdersInLastYear, boolean  
isSubscribedToNewsletter)
```

In this project, we followed the **Model-Driven Test Design** method.

This means that we designed the test cases first before writing the code. We first looked at the inputs of the system, divided them into blocks, checked if there are any impossible combinations, and then created Pair-Wise test cases.

This helps us test the program in an organized way instead of choosing random test cases.

Input Space Partitioning (ISP)

In this project, we applied Input Space Partitioning (ISP) as part of the MDTD methodology. First, we identified the main characteristics that affect the behavior of the discount calculation system. Then, each characteristic was divided into blocks that represent possible groups of input values.

Characteristics and Blocks

Characteristic C1: Customer Type

- Block B1: NEW
- Block B2: REGULAR
- Block B3: PREMIUM

Characteristic C2: Total Orders in Last Year

- Block B1: Less than 10 orders
- Block B2: 10 orders or more

Characteristic C3: Newsletter Subscription

- Block B1: Subscribed
- Block B2: Not Subscribed

The purpose of dividing the inputs into blocks is to reduce the number of needed test cases while still covering different possible behaviors of the system.

Pair-Wise Coverage (PWC)

After identifying the characteristics and their corresponding blocks, Pair-Wise Coverage (PWC) was applied to generate test combinations. The goal of PWC is to ensure that every possible pair of blocks between two characteristics appears in at least one test case.

Pair-Wise Combinations

Test Case	C1: Customer Type	C2: Total Orders	C3: Newsletter Subscription
TC1	B1 (NEW)	B1 (<10)	B1 (Subscribed)
TC2	B1 (NEW)	B1 (<10)	B2 (Not Subscribed)
TC3	B2 (REGULAR)	B1 (<10)	B1 (Subscribed)
TC4	B2 (REGULAR)	B2 (≥10)	B2 (Not Subscribed)
TC5	B3 (PREMIUM)	B1 (<10)	B2 (Not Subscribed)
TC6	B3 (PREMIUM)	B2 (≥10)	B1 (Subscribed)

Infeasible Combination

Test Case	C1: Customer Type	C2: Total Orders	Reason
TC7	B1 (NEW)	B2 (≥10)	A NEW customer cannot have 10 or more orders in the last year

This infeasible combination should result in an `IllegalArgumentException` according to the system requirements.

Parameterized Testing

In this project, we implemented **Parameterized Testing** using JUnit 5. By utilizing the **@ParameterizedTest** annotation alongside **@CsvSource**, we were able to execute multiple test scenarios through a single method, which avoids duplicating the code.

Each entry within the CsvSource corresponds to a specific Pair-Wise test case derived from our ISP model. This methodology enhanced several aspects of our project:

- Readability
- Maintainability
- Test organization
- Testing efficiency

The use of parameterized testing simplified the process of validating various customer scenarios in a systematic manner while minimizing redundant test logic.

Test Automation and Implementation

Subsequent to the design of our Pair-Wise test scenarios, we developed automated tests utilizing **JUnit 5 Parameterized Testing**.

The development cycle adhered strictly to the **Red-Green-Refactor** stages inherent to the **Model-Driven Test Design (MDTD)** methodology.

Initially, test cases were authored and executed prior to the system logic implementation, resulting in expected initial failures (**Red phase**). We then incrementally introduced the requisite logic until all scenarios achieved a passing status (**Green phase**). Ultimately, the codebase underwent refactoring to optimize clarity and maintainability while preserving functional integrity (**Refactor phase**).

The implementation scope encompassed:

- base discount calculation
- newsletter subscription bonus
- customer type bonus
- loyalty bonus
- infeasible combination validation
- maximum discount cap handling

Limitations of Pair-Wise Testing

While **Pair-Wise Testing** significantly optimizes the test suite by reducing the volume of scenarios, it cannot ensure the detection of every defect. Certain logic errors may only manifest through the simultaneous interaction of three or more input parameters.

Further constraints include:

- hidden dependencies between parameters
- missing edge cases
- additional business-rule complexity
- the need for extra validation testing

Consequently, this methodology should be integrated with complementary testing techniques to maximize **software reliability** and functional coverage.

Team Contribution Matrix

Team Member	Contributions
Rana	Created the Maven project setup, added JUnit dependencies, created <code>DiscountCalculator.java</code> and <code>DiscountCalculatorTest.java</code> , prepared the Input Space Partitioning (ISP) analysis, identified characteristics and blocks, prepared the Pair-Wise Coverage (PWC) combinations, and created the initial parameterized test structure.
Maria	Completed the parameterized tests with assertions, added infeasible combination tests, implemented the discount calculation logic, finalized and executed the tests, wrote the report documentation sections, and finalized the <code>ProjectReport.pdf</code> formatting.